

Commit 7a0765b

Browse files

ActuallyNull committed last month

model v1.4, data_preperation now obsolete, all included in data_processing.py; improved data processing algorithm (all in one -> batch processing), added more regularisation to the model (added more dropouts and l2 reg) and class_weights

main 1 parent 849e037 commit 7a0765b

- data_preperation.py
- data_processing.py
- model.py

3 files changed +57 -21 lines changed

data_preperation.py -14

```
... @@ -1,14 +0,0 @@
1 - from data_processing import preprocess_dataset
2 - from sklearn.preprocessing import LabelEncoder
3 - from sklearn.model_selection import train_test_split
4 -
5 - def create_train_val_test_splits():
6 -     X = []
7 -     y = []
8 -     X_processed, y_processed = preprocess_dataset("training2017/training2017", "training2017/training2017/REFERENCE.csv",
9 - X, y)
10 -     X_cnn = X_processed.reshape((X_processed.shape[0], X_processed.shape[1], 1))
11 -     le = LabelEncoder()
12 -     y_cnn = le.fit_transform(y_processed)
13 -     X_train, X_temp, y_train, y_temp = train_test_split(X_cnn, y_cnn, test_size=0.3, random_state=42, stratify=y_cnn)
14 -     X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp)
15 -     return X_train, y_train, X_val, y_val, X_test, y_test
```

data_processing.py +43 -4

```
... @@ -3,9 +3,12 @@
3 import os
4 import pandas as pd
5 from sklearn.preprocessing import MinMaxScaler
6 - from scipy.signal import butter, filtfilt, resample
7 + from scipy.signal import butter, filtfilt
8
9 from imblearn.over_sampling import SMOTE
10 from ecg_augmentor import ECG_Augmentor
11 + from sklearn.preprocessing import LabelEncoder
12 + from sklearn.model_selection import train_test_split
13 + import gc
14
15 physionet_label_map = {'N': "Normal", 'A': "AFib", 'O': "Other Arrythmia"}
16 duration = 10 # seconds
17
18 @@ -70,11 +73,47 @@ def load_physionet_dataset(physionet_path, reference_file_path, X, y):
70 return X, y
71
72 def preprocess_dataset(physionet_path, reference_file_path, X, y, fs=300):
73 -     augmentor = ECG_Augmentor(fs=fs)
74     X, y = load_physionet_dataset(physionet_path, reference_file_path, X, y)
75     X_filtered = np.array([butter_bandpass_filter(x, fs=fs) for x in X])
76 -     X_augmented = np.array([augmentor.augment(x) for x in X_filtered])
77 -     X_processed = normalise_dataset(X_augmented) # For DCNN, normalization is preferred
78 +     X_processed = normalise_dataset(X_filtered) # For DCNN, normalization is preferred
79     y_processed = y
80     X_balanced, y_balanced = smote_resampling(X_processed, y_processed)
81     return X_balanced, y_balanced
82 +
83 + def create_train_val_test_splits(fs=300):
84 +     X = []
85 +     y = []
86 +     augmentor = ECG_Augmentor(fs=fs)
87 +     # Load and preprocess only, avoid unnecessary copies
88 +     X_processed, y_processed = preprocess_dataset(
89 +         "training2017/training2017",
90 +         "training2017/training2017/REFERENCE.csv",
91 +         X, y
92 +     )
93 +     # Use float32 to save memory
94 +     X_processed = X_processed.astype(np.float32)
95 +     # Reshape for CNN
96 +     X_cnn = np.expand_dims(X_processed, axis=-1)
97 +     # Encode labels
98 +     le = LabelEncoder()
99 +     y_cnn = le.fit_transform(y_processed)
100 +     # Split data
101 +     X_train, X_temp, y_train, y_temp = train_test_split(
102 +         X_cnn, y_cnn, test_size=0.3, random_state=42, stratify=y_cnn
103 +     )
104 +     X_val, X_test, y_val, y_test = train_test_split(
105 +         X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp
106 +     )
107 +     # Augment only training data, process in batches to save RAM
108 +     batch_size = 500 # Adjust as needed for your RAM
109 +     X_train_aug = []
110 +     for i in range(0, len(X_train), batch_size):
111 +         batch = X_train[i:i+batch_size]
112 +         batch_aug = [augmentor.augment(x.squeeze()) for x in batch]
113 +         batch_aug = np.expand_dims(np.array(batch_aug, dtype=np.float32), axis=-1)
114 +         X_train_aug.append(batch_aug)
115 +     X_train_aug = np.concatenate(X_train_aug, axis=0)
116 +     # Clean up memory
117 +     del X, y, X_processed, X_cnn, X_train, X_temp, batch, batch_aug
118 +     gc.collect()
119 +     return X_train_aug, y_train, X_val, y_val, X_test, y_test, le
```

model.py +14 -3

```
... @@ -1,13 +1,19 @@
1 import tensorflow as tf
2 from keras import layers, models
3 from keras.callbacks import EarlyStopping, ReduceLROnPlateau
4 - from data_preperation import create_train_val_test_splits
5 + from data_processing import create_train_val_test_splits
6
7 from sklearn.metrics import confusion_matrix, classification_report
8 import matplotlib.pyplot as plt
9 from sklearn.metrics import RocCurveDisplay
10 from sklearn.preprocessing import label_binarize
11 + from sklearn.utils.class_weight import compute_class_weight
12 + from keras import regularizers
13 + import numpy as np
14
15 - X_train, y_train, X_val, y_val, X_test, y_test = create_train_val_test_splits()
16 + X_train, y_train, X_val, y_val, X_test, y_test, le = create_train_val_test_splits()
17 +
18 + weights = compute_class_weight('balanced', classes=np.array([0,1,2]), y=y_train)
19 + class_weights = dict(enumerate(weights))
20
21
22 callback = [
23     EarlyStopping(monitor='val_loss', patience=7, verbose=1, restore_best_weights=True),
24 @@ -21,23 +27,27 @@ def ecg_cnn(input_length, num_classes=3): # 0: afib, 1: normal, 2: other arrhyth
21 x = layers.BatchNormalization()(x)
22 x = layers.ReLU()(x)
23 x = layers.MaxPooling1D(pool_size=2)(x)
24 + x = layers.Dropout(0.2)(x)
25
26 x = layers.Conv1D(64, kernel_size=11, padding='same')(x)
27 x = layers.BatchNormalization()(x)
28 x = layers.ReLU()(x)
29 x = layers.MaxPooling1D(pool_size=2)(x)
30 + x = layers.Dropout(0.2)(x)
31
32 x = layers.Conv1D(128, kernel_size=7, padding='same')(x)
33 x = layers.BatchNormalization()(x)
34 x = layers.ReLU()(x)
35 x = layers.MaxPooling1D(pool_size=2)(x)
36 + x = layers.Dropout(0.2)(x)
37
38 x = layers.Conv1D(256, kernel_size=5, padding='same')(x)
39 x = layers.BatchNormalization()(x)
40 x = layers.ReLU()(x)
41 x = layers.GlobalAveragePooling1D()(x)
42 + x = layers.Dropout(0.2)(x)
43
44 x = layers.Dense(64, activation='relu')(x)
45 + x = layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.001))(x)
46
47 x = layers.Dropout(0.5)(x)
48 outputs = layers.Dense(num_classes, activation='softmax')(x)
49
50 @@ -55,6 +65,7 @@ def ecg_cnn(input_length, num_classes=3): # 0: afib, 1: normal, 2: other arrhyth
55 epochs=100,
56 batch_size=64,
57 callbacks=callback,
58 + class_weight=class_weights
59 )
60
61 test_loss, test_acc = model.evaluate(X_test, y_test)
```

Comments 0

Please sign in to comment.